

Aakash Business ERP

Developer Guide

Architecture, posting logic and module reference

Edition: 29 September 2026

Contents

1. Architecture
2. Document life cycle (all transactions)
3. Accounting engine
4. Inventory engine
5. Source documents and progress
6. Billing terms
7. Numbering, masters, currency
8. Settings that change behaviour
9. Reports
10. Industry modules
11. Adding a new transaction module - checklist
12. Running and testing locally

This guide explains how the system is built, where each piece of logic lives, and exactly how every transaction affects the accounts (general ledger) and the inventory (stock). Keep it next to the code: when you change a posting rule, update the matching section here.

A user-level description of every menu is in `docs/USER_MANUAL.md`. Printable copies of both are in `docs/pdf/` (build them with `python3 docs/tools/build_pdfs.py`).

1. Architecture

1.1 Stack

- **Database:** PostgreSQL through Supabase. All company data lives in the `tenant_master` schema. Every row carries `tenant_id`, so one database serves many companies.
- **Server:** Node.js + Express (`server/`). `server.js` mounts one router per module from `server/routes/`; shared business logic lives in `server/utils/`.
- **Client:** React 18 (Create React App) with Tailwind CSS, moving to TypeScript file by file (`client/src/`). Pages in `client/src/pages/`, shared entry parts in `client/src/components/entry/`, the menu in `client/src/components/menu.ts`.
- **Migrations:** numbered SQL files in `database/` (`001_...sql` to `141_...sql`). Every migration is idempotent (`IF NOT EXISTS, DO $$ ... EXCEPTION WHEN duplicate_object`) so it can be run again safely.

1.2 Request flow

1. The client calls `authFetch('/api/...')` from `contexts/AuthContext.jsx`, which adds the JWT.
2. `middleware/auth.js requireAuth` checks the token and sets `req.auth = { userId, tenantId, isSuperAdmin, ... }`. `utils/dataAccess.guard` then runs for every call (see 8.3).
3. `loadUserPermissions + requirePermission(module, action)` check the Security Rights Group.
4. The route gets a tenant client with `getTenantClient(tenantId)` (`utils/dbHelpers.js`) and always filters by `tenant_id`.
5. Changes are logged with `logAudit` (activity log) and, for documents, `logDocumentAudit` (per-document audit trail). Database triggers (`audit_attach_all`) also record old/new values for the Audit Log screen.

1.3 Folder map

Path	What is in it
<code>server/server.js</code>	Express app, router mounting, the approval guard on <code>PUT /api/:api/:id/status</code>
<code>server/routes/*Routes.js</code>	One file per module: list, get, create, update, status, delete, pull-forward
<code>server/utils/</code>	Posting, stock, numbering, terms, pricing, reports, IRD, messaging engines
<code>server/middleware/</code>	Auth, permissions
<code>database/*.sql</code>	Schema and migrations, in order
<code>client/src/pages/</code>	One page per menu item
<code>client/src/components/entry/</code>	Shared transaction parts: line grid, term popups, fill bar, actions, currency field, amount cell
<code>client/src/components/menu.ts</code>	The whole navigation (TOP_MENUS and REPORT_GROUPS)
<code>client/src/hooks/</code>	Entry settings, entry field controls, Enter-key navigation, master codes, features
<code>r5/</code>	Server unit tests (mock DB): <code>node r5/<name>_test.js</code>

2. Document life cycle (all transactions)

Every transaction table has a **status** column. The common statuses are:

Status	Meaning	Ledger / stock effect
(draft in <code>entry_drafts</code>)	Save as Draft - kept outside the module table	None
draft	Saved, waiting for approval (only when the module needs approval)	None
posted / received / confirmed / approved	Live document	Posted
cancelled	Cancelled with a reason	Everything reversed

2.1 Save, Save as Draft, approval

- **Save as Draft** (`client/src/components/entry/entryDrafts.js`): the typed form is stored as JSON in `tenant_master.entry_drafts` (migration 137) with `POST /api/entry-drafts`. It gets no number and never touches the module table, ledger or stock. Opening it from the fill bar remembers its id (`setCurrentDraft`); when the real document is saved, `finishEntryDraft` deletes it.
- **Save**: the page calls the module's create/update route and then `finalizeEntry(authFetch, api, id, status)` (`components/entry/DocActions.jsx`). `finalizeEntry` reads the approval policy (`GET /api/document-actions/policy`):
 - Module **not** in System Control > Approval Needed For: it immediately calls `PUT /api/<api>/:id/status` with the live status, so the document posts on Save.
 - Module **in** the list: the document stays **draft** with an "Awaiting approval" tag. A user with the approval right presses Approve/Post, which calls the same status route.
- **Server enforcement** (`server/utils/approval.js`): `guard` is mounted before the routers on `PUT /api/:api/:id/status`. `API_TYPES` maps each API to its document type and live statuses. When the type is in `system_control_settings.approval_modules`, only a super admin or a user whose group has `permissions.approvals[<type>] = true` may move it to a live status; anyone else gets 403.

2.2 Posting happens in the status route

Creating or editing a document only stores it. **All ledger, stock, bill-wise and progress effects happen in `PUT /api/<api>/:id/status`**, guarded by the previous status so posting twice never doubles anything:

```
if (status is live && existing.status is not live) -> post GL, stock, bill-wise, progress (+)
if (status === 'cancelled' && existing.status was live) -> reverse GL, stock, bill-wise, progress (-)
```

Cancel needs a `cancellation_reason`. A posted document settled bill-wise against another document cannot be cancelled until that settlement is removed (`checkCanCancelIfSettled`).

2.3 Modify, Remove, Cancel, Reverse, Copy

`components/entry/DocActions.jsx` renders the action bar on every transaction: Create, Modify, Remove, Cancel, Save as Draft, Print.

- **Modify** a draft simply opens it. Modify on a posted document first moves it through the module's status route, which reverses its ledger and stock effect, and reopens it as a draft with the same number. The user changes it and posts again. Sales Return shows only Reverse.
- **Remove** (hard delete) is only allowed while the document is a **draft**.
- **Cancel** keeps the record with status **cancelled** and reverses every effect.

- **IRD Reverse:** when Computerized Billing (IRD) is on in System Control, a posted Sales Bill or Sales Return cannot be edited or cancelled in place. It is reversed by a return document instead, and the action is labelled Reverse. `utils/ird.js` keeps the IRD materialized register and pushes to CBMS.

2.4 Fill from Template / Previous / Draft

`components/entry/EntryFillBar.jsx` sits on top of every entry screen.

- Templates: `GET/POST/DELETE /api/entry-templates` (table `entry_templates`, migration 136). They can be personal or shared.
- Previous: the screen's own list (`GET /api/<api>`); `asNewCopy` drops ids, numbers and status so the copy saves as a new document.
- Drafts: see 2.1.

3. Accounting engine

3.1 Where entries are stored

Every posted document writes one or more **batches**:

- `ledger_transaction_batches` (`document_type`, `document_id`, `batch_date`, `narration`)
- `ledger_transaction_lines` (`ledger_account_id`, `sub_ledger_id`, `product_company_id`, `debit_amount`, `credit_amount` + dimensions such as `cost center`, `business unit`, `branch`)

`utils/grnAccounting.js` holds `postBatch` and `reverseBatch(documentType, documentId)`, which deletes all batches of a document. Every batch must balance (sum of Dr = sum of Cr); the route refuses to post if it does not.

All reports (Ledger, Trial Balance, P&L, Balance Sheet, Day Book, Party Summary) read only these two tables plus ledger opening balances, through `utils/financialEngine.js`.

3.2 Which ledger each line goes to

`utils/accountResolver.js` `splitByAccount` splits a document's VAT-exclusive value per line:

1. The product's own account (`products.sales_account_ledger_id / purchase_account_ledger_id` with its sub-ledger). A return uses the product's Return Account with the Return Sub-Ledger; a non-saleable return uses its Non-saleable Return Account and sub-ledger when set. With no return account, the main account is used with the return sub-ledger (else the main sub-ledger). Product save checks each sub-ledger belongs to its account.
2. The document's account (`sales_account_ledger_id / goods_account_ledger_id` + sub-ledger chosen on the entry).
3. The System Control default (`sales_[return_]account_ledger_id / purchase_[return_]account_ledger_id`).

Document-level terms that are not VAT (freight, bill discount) go to the document-level account (2, then 3), or to the term's own ledger when the billing term has one. A line that resolves to no ledger blocks posting.

`utils/ledgerPurpose.js` `checkAccountPurposes` makes sure a goods account is a P&L (trading) ledger or an inventory ledger, never a random balance-sheet ledger.

3.3 VAT and billing-term ledgers

`utils/vatLedger.js` is the one rule for VAT:

- VAT computed by a billing term with `tax_type = 'vat'` goes to that term's ledger.
- Line VAT (`tax_amount`) goes to the System Control default VAT ledger (`defaultVatLedger`).
- The same function feeds the VAT reports, so the GL and the VAT register never differ.

Other billing terms post to the ledger set on the term (Billing Terms master), with the sub-ledger chosen in the entry popup (`utils/termSubLedgers.js`: Return Sub-Ledger on returns, Billing Sub-Ledger elsewhere). Excise terms (`tax_type = 'excise'`) post to their ledger and are reported by `exciseOf`.

3.3c Stock cost of purchases, landed cost, non-stock items (round 16)

- `utils/purchaseStockCost.js`
 - `lineUnitCosts`: GRN / direct Bill / Purchase Return lines enter stock at their share of the document's goods value without VAT (what the purchase account gets) divided by the base-unit qty.
 - `refreshLandedCost`: adds the allocated amount of every **posted** additional expense (`purchase_expense_allocations`) to the cost of the stock receipt of that line. The link can be the GRN

line, the Bill line (or the GRN line it came from), or an Order line (split by qty).

- `stock_movements.base_unit_cost` keeps the cost without landed cost. Posting or cancelling an expense, or re-posting the GRN / Bill, recomputes it.
- `utils/stockItems.js`: item types `service`, `non_inventory` and `fixed_asset` write no stock movement.
- `utils/negativeStock.js`: one negative-stock rule for delivery, direct sales bill and purchase return. Lines are added up per item, warehouse and batch, in base units.
- Stock engine (`stockEngine` / `stockReport`): sales returns come back at cost, never at the selling rate.
- Production output and by-products: total cost / base qty.
- Migrations:
 - 145: re-costs existing receipts and production.
 - 146: DB trigger - no GL batch or stock movement is added to or removed from a closed / locked fiscal year.
 - 147: removes stock movements of non-stock items in open years.
- Purchase bill: the same supplier bill no cannot be entered twice for one supplier (`duplicatePartyBill`, on save and on posting).
- `authFetch` keeps the server's extra error fields (`warnings`, `credit_blocked`), so "Post anyway?" and the credit override work.

3.3f Day book, write-off, module filter, batch valuation (migration 150)

- `utils/dayBook.js` (`/api/day-book`, `/types`): reads the documents (not the GL) so it can filter by `created_by` and `agent_id` (multi-line cash / bank entries by line agent); cash / credit per voucher (sales bill `received_amount` counts as cash); cash & bank from the GL lines of the shown vouchers on cash / bank ledgers (opening / closing only when not narrowed); party summary with closing balance.
- `utils/balanceWriteoff.js` (`/api/balance-writeoff/preview|post|register`): one JV `jv_type` = `'balance_writeoff'` through the JV handlers; `balance_writeoff_lines` per party carry the bill-wise settlement (`source_type` `'balance_writeoff'`); JV cancel calls `reverseWriteoffSettlements`.
- `utils/dimensionReports.js`: `modules` filter (`moduleOf(doc_type)`); a pivot with rows ledger + sub-ledger returns `subtotals` per ledger (master opening included).
- `utils/stockValuation.js`: `methodFor` / `keyEvents` per product like `stockMovement` - batch / serial costing of System Control; `rows[].costing` names the method used. Landed cost lives in `stock_movements.unit_cost`, so batch-wise layers and Profitability (`costRatesOn`) carry it.

3.3e Salesman mobile collection / returns, bill-wise commission (migration 149)

- `utils/mobileEntries.js`: `createMobileReceipt` (customer must be on a route planned for the salesman that day unless `allow_off_route_orders`; cash ledger = `salesman_agents.mobile_cash_ledger_id` or System Control `default_cash_ledger_id`) and `createMobileReturn` (customer of any route). Both run the screens' own create handlers (`handlerOf(router, method, path) + invoke`), stay `draft`, and are marked `entry_source = 'mobile'` with `agent_id` = the salesman.
- `pendingEntries` / `postEntries` (`/api/mobile-entries/pending`, `/post`): the office ticks and posts; posting runs the status handlers (GL, stock, bill-wise, IRD as usual).
- `utils/salesman.agentParties`: only customers of the routes planned for the day (`routeIdsFor`), ordered by route then `sequence_order`; off-route salesmen also get their other routes. `areasRoutes`, `anyRouteCustomers` for returns.
- `utils/agentBillCommission.js`: `commissionBills` (posted bills with an agent: value excl. VAT less posted returns against the bill, agent %, paid state) and `postBillCommission` (one posting per agent, `basis` =

'bill', rows in `agent_commission_bills`; unique index `ux_commission_bill_live` keeps a bill in one live posting). `agentTargets.cancelPosting` sets the bills inactive so they can be paid again.

- `utils/loadingSheet.js`: `qty_totals` per block - base qty per base unit, entered qty per unit, free.

3.3d Additional-bill terms (round 17, migration 148)

- `billing_terms.applicable_sales_additional + applicable_additional_expense` = "Additional term"; `applicable_sales_entry / purchase_entry / production_entry` = "Transaction term". `include_in_costing` (default true) next to `include_in_profitability`.
- `purchase_additional_expense_lines`: `billing_term_id`, `expense_sub_ledger_id`, `target_detail_id`, `include_in_profitability`.
- `purchaseAdditionalExpenseRoutes.applyTermRules` (save and preview): the term's ledger replaces the line ledger; sub-ledger not sent = the term's default; a term with `include_in_costing = false` gets basis none.
- `computeAllocations`: a line with `target_detail_id` (product-wise term) puts its whole amount on that Order / GRN / Bill line; other lines are split by their basis. An unknown target falls back to the basis.
- `grnAccounting.buildAdditionalExpenseGl`: Dr / Cr the term ledger with `expense_sub_ledger_id`; the party side uses the line party + `party_sub_ledger_id` (or the entry vendor + its sub-ledger).
- Client (`PurchaseAdditionalExpense.jsx`): tab Product-wise (popup per product) and tab Bill-wise (one row per bill-level term, pre-filled on a new entry). Linking a Bill / GRN / Order fills its vendor.
- Page shell (`Layout.jsx`): the page tools sit on the window title bar (Related / Reports are drop-downs); `EntryFillBar` is one line with a "Template " menu.

3.3b Reconciliation of registers and books (`utils/taxReconciliation.js`)

GET `/api/vat-reports/reconciliation?section=vat|sales|purchase|tds|all&date_from&date_to&ledger_ids=` (page `TaxReconciliation.jsx`, `/tax-reconciliation`).

- Register side: `loadTaxDocs` (the same loader as the VAT reports) plus non-saleable returns; TDS from bills, JVs (`tds_amount`, `tds_side`), JV lines with `tds_percent` and expense-bill deduct lines.
- Books side: `ledger_transaction_lines` of the section's ledgers joined to their batch, grouped by (`document_type`, `document_id`). A purchase bill with `source_grn_id` is grouped with its GRN (goods are booked at GRN time).
- Row status: `matched / difference / register_only / books_only`. The books column adds up to the whole movement of those ledgers, so the summary ties to the ledger report.
- Ledger sets (`sectionLedgers`): VAT = `allVatLedgerIds`; sales = System Control + product sales / return / non-saleable accounts + document accounts + JV sales lines; purchase = the purchase equivalents + expense-bill ledgers + JV purchase lines; TDS = System Control TDS ledgers + ledgers used on bills / JVs / expense deduct lines.

3.3a TDS and money received on the bill (`utils/billExtras.js`, migration 142)

- **TDS** on Sales Bill and Purchase Bill: `tds_percent`, `tds_base_amount` (default: VAT-exclusive bill value), `tds_amount` (default: base x %), `tds_ledger_id` (default: System Control `sales_tds_ledger_id` for sales, `tds_ledger_id` for purchase).
- **Receipts** on Sales Bill: `receipts` JSON [{ `ledger_id`, `sub_ledger_id`, `amount`, `ref_no` }]; every ledger must be a Cash / Bank (or overdraft) ledger (`ledgerPurpose` purpose `cash_bank`).

- Cash bill: received + TDS must equal the bill total. With no lines, the whole amount goes to System Control `default_cash_ledger_id`.
- Credit bill: part payment allowed; the balance stays outstanding.
- `prepareExtras` runs before the status changes (a problem returns 400 and nothing posts); `gLlLines` adds the lines to the bill's voucher; `settleOnBill` makes one extra bill-wise reference settled against the bill; `unsettleOnBill` removes it on cancel. `checkCanCancelIfSettled` ignores a document's own settlement.
- TDS Report (`/vat-reports/tds`) lists JV TDS lines, additional-expense TDS lines and bills with TDS.

3.4 Posting rules per transaction

Dr = debit, Cr = credit. "Goods account" means the per-line result of 3.2.

Document (route)	Posts when	Accounting entry	Stock movement	Bill-wise reference
Sales Quotation (<code>salesQuotationRoutes</code>)	sent / accepted	None	None	None
Sales Order (<code>salesOrderRoutes</code>)	confirmed	None (credit limit checked)	None	None
Sales Delivery / Challan (<code>salesDeliveryRoutes</code>)	posted	None	OUT from warehouse; order <code>qty_delivered</code> +	None
Sales Bill (<code>salesBillRoutes</code>)	posted	Dr Customer (grand total); Cr Sales account per line; Cr VAT; Cr/Dr term ledgers. Same voucher: TDS by customer (Dr TDS receivable, Cr Customer) and money received with the bill (Dr each cash / bank, Cr Customer)	OUT only for lines not pulled from a delivery (direct bill)	<code>dr</code> reference for the customer; TDS + receipts settle it (<code>sales_bill_settle</code>)
Sales Return (<code>salesReturnRoutes</code>)	posted	Dr Sales Return account per line; Dr VAT; Cr Customer	IN to warehouse; bill <code>qty_returned</code> +	<code>cr</code> reference, settles open <code>dr</code> bills FIFO
Sales Non-saleable Return	posted	<code>credit_note</code> settlement: like Sales Return using the non-saleable return account. <code>no_credit</code> : none	IN to the separate nonsaleable_stock_movements ledger, never main stock	<code>cr</code> when credited
Sales Additional Entry	posted	Dr Customer (net); Cr income ledger per <code>add</code> line; Dr income ledger per <code>deduct</code> line	None	<code>dr</code> reference
Purchase Requisition / Quotation / Order	approved / accepted / confirmed	None	None	None
Purchase GRN (<code>purchaseGrnRoutes</code>)	received	Dr Goods account (VAT-inclusive); Cr GRN Clearing (only when System Control has a GRN Clearing ledger)	IN to warehouse; order <code>qty_received</code> +	None
Purchase Bill (<code>purchaseBillRoutes</code>)	posted	Direct bill: Dr Goods per line (VAT-exclusive), Dr VAT, Cr Vendor. From GRN: Dr GRN Clearing, Cr Vendor, and the VAT moved out of the goods account into the VAT ledger. TDS: Dr Vendor, Cr TDS payable (own batch)	IN only for lines not pulled from a GRN	<code>cr</code> reference; clears any <code>dr</code> advance FIFO first; TDS settles it (<code>purchase_bill_settle</code>)
Purchase Return	posted	Dr Vendor; Cr Goods / return account per line; Cr VAT	OUT; bill <code>qty_returned</code> +	<code>dr</code> reference, settles open <code>cr</code> bills FIFO

Document (route)	Posts when	Accounting entry	Stock movement	Bill-wise reference
Purchase Non-saleable Return	posted	write_off : Dr Write-off Expense, Cr Non-saleable Stock Asset. credit_note : like Purchase Return	OUT of nonsaleable_stock_movements	dr when credit note
Purchase Additional Expense	posted	add line: Dr expense ledger (+VAT if VAT is part of cost), Dr VAT (claimable), Cr party. deduct line (e.g. TDS, is_tds): Cr that ledger (TDS payable), Dr party - a TDS line is never costed. Parties are netted.	None directly; the amount is allocated to the linked Order/GRN/Bill lines for landed cost (value, qty or equal share)	None
Journal Voucher	posted	The lines as typed (Dr total must equal Cr total). jv_type (migration 143, JV_TYPES in journalVoucherRoutes.js) : goods / asset / service purchase or sales tie party (total less TDS), VAT and TDS lines to the header figures and check the account by purpose (purchase_goods/expense , fixed_asset , pl_expense , sales_goods , income); tds with tds_side purchase / sales against bills (utils/jvTdsBills.js , table jv_tds_bills , migration 144): purchase Dr supplier / Cr TDS payable, sales Dr TDS receivable / Cr customer; bills re-read and priced from the database (base = value without VAT), one TDS per bill (a bill on a journal that is not cancelled, or with its own TDS, is not offered); without bills (purchase) Dr expense, Cr TDS payable, Cr party (base less TDS). Party must be of the side's category (purchase/both or sales/both). tax_entry_type stays purchase / sales for the VAT register.	None	Party lines create dr/cr references
Cash / Bank Entry	posted	Receipt: Dr Cash/Bank, Cr each line ledger. Payment: Dr each line ledger, Cr Cash/Bank	None	Receipt = cr (settles dr); Payment = dr (settles cr)
Debit Note	posted	Dr party; Cr the ledger lines	None	dr reference
Credit Note	posted	Dr the ledger lines; Cr party	None	cr reference
PDC	realized only	Received: like a receipt. Issued: like a payment. Nothing at creation or deposit	None	At realization
Bulk Cash Settlement	posted	Many receipts in one run	None	Per party

Document (route)	Posts when	Accounting entry	Stock movement	Bill-wise reference
Stock Transfer (stockTransferRoutes)	approved / posted	Only when System Control Stock Transfer GL Posting is on: Dr receiving stock a/c, Cr sending stock a/c. Two-step branch transfer: Dr Goods in Transit, Cr sending; then on receipt Dr receiving, Cr Goods in Transit	OUT from sending warehouse, IN to receiving	None
Stock Adjustment (stockAdjustmentRoutes)	posted	Decrease: Dr Stock Shortage/Damage (expense), Cr Stock Adjustment. Increase: Dr Stock Adjustment, Cr Stock Excess (income)	IN or OUT	None
Production Order (productionOrderRoutes)	posted	None (cost moves through stock valuation)	Raw materials OUT; finished goods and by-products IN at cost (joint cost allocation in utils/jointCostAllocation.js)	None
Depreciation (utils/fixedAssets.js)	run posted	Dr Depreciation expense, Cr Accumulated depreciation (or the asset ledger)	None	None
Asset disposal	on sale / scrap	Dr Accumulated depreciation, Dr Disposal ledger (book value), Cr Asset ledger (cost). A Sales Bill of an asset product disposes it automatically.	None	None
Interest on overdue (utils/interest.js)	run posted	Dr each customer, Cr Interest income (unpaid x rate% x days / 365 per overdue bill, FIFO-settled; never charged twice)	None	None
Ledger Opening Balance	saved	Opening balance on the ledger (not a batch)	None	Opening references when bill-wise
Product Opening Stock	saved	None (opening stock value is read by the stock engine)	Receipt on the day before the first fiscal year	None

3.5 Why stock and GL agree (periodic stock)

The system uses **periodic stock** accounting ([utils/financialEngine.js](#), [utils/stockAccounting.js](#)):

- Stock quantity and value come from [stock_movements](#) through [utils/stockEngine.js](#), valued by the method chosen in System Control (FIFO, moving average, weighted average, last purchase; batch/serial costing when enabled).
- The Balance Sheet shows Closing Stock from the stock engine instead of the GL balance of Inventory-group ledgers.
- Profit is **GL (Income - Expenses) + Closing Stock - Inventory GL balance**, so **COGS = Opening Stock + Purchases + Inventory-account movement - Closing Stock**.
- Because of this, a Sales Bill does not post a COGS entry. The cost of goods sold appears through the closing stock. Stock transfers and adjustments may post only to "stock-neutral" ledgers (inventory group or purchase/direct expense group), or to a clearly labelled loss/gain ledger; [stockAccounting.js](#) refuses other ledgers.

3.6 Bill-wise settlement

[utils/billWiseSettlement.js](#). When a party (or System Control) has bill-wise tracking on:

- Each posted document creates a reference with nature **dr** (sales bill, debit note, payment) or **cr** (purchase bill, credit note, receipt, return).
- **getOutstandingReferences** lists the opposite-nature outstanding items oldest first; **computeFifoAllocation** consumes them in FIFO order. The entry screen shows this suggestion in **BillwiseSettlementPanel** and the user may adjust it.
- Whatever is left becomes the new document's own outstanding amount.
- Cancelling calls **reverseReferenceAndSettlements**. When Product Company is compulsory, settlement is limited to the same product company.

Outstanding, Ageing, Bill-wise Ageing and Interest reports read these references.

3.7 Credit control

utils/creditControl.js resolves the credit limit (ledger override, else System Control default) and checks it on Sales Order and Sales Bill. The action may be warn or block, depending on settings.

4. Inventory engine

4.1 Stock movements

`stock_movements` holds one row per product per warehouse per document line:

- `source_type`, `source_id`, `product_id`, `warehouse_id`, `batch/serial`, `qty_in / qty_out` in the **base unit**, `rate`, `value`, `movement_date`.
- Posting writes rows; cancelling deletes rows of that source (`reverse...StockMovements`).
- Units are converted to the base unit by `utils/unitConversion.toBaseUnitQty`, or, for dual-unit items, by `dualUomCalculation.toBaseQtyFromDual`.
- Negative stock control (System Control) is checked before OUT movements.

4.2 Which document moves stock

IN	OUT
Product opening stock	Sales Delivery
Purchase GRN	Sales Bill (direct lines only)
Purchase Bill (direct lines only)	Purchase Return
Sales Return	Production raw material
Production output / by-products	Stock Transfer (sending side)
Stock Transfer (receiving side)	Stock Adjustment decrease
Stock Adjustment increase	

Non-saleable returns use `nonsaleable_stock_movements` so damaged goods never mix with saleable stock.

4.3 Valuation

`utils/stockEngine.js` gives, for a period and method, per item: Opening + In - Out = Closing, for both quantity and value. It is used by the Stock Movement report and by the financial statements, so both always match.

`utils/stockValuation.js` provides batch/serial costing and the valuation report. Additional expenses allocated to purchase lines raise their landed cost.

4.4 Dual unit of measure

`utils/dualUomCalculation.js`:

- **Fixed dual** (`fixed`): primary + secondary quantities are added (5 CRT + 3 PCS).
- **Flexible** (`auto_convert`): the secondary field holds the total in the secondary unit.
- **Rate basis**: `products.dual_rate_basis` (migration 137) is `any` (the line chooses whether the rate is per primary or per secondary unit), `primary` or `secondary` (fixed). `rateBasisFor(client, productId, lineBasis)` enforces it on the server; `dualUomEntryMode.fixedRateBasis` locks it in the grid.
- Progress counters also move the alternate-unit counter (`progressCounters.bumpAltCounter`).

5. Source documents and progress

5.1 Pull forward

A later document can be filled from earlier ones (`utils/pendingDocs.js`, `PendingDocsPanel.jsx`). Which sources are offered is controlled by Entry Field Control:

Target	Sources
Sales Order / Purchase Order	Quotation
Sales Delivery / Purchase GRN	Quotation + Order
Sales Bill	Quotation + Order + Delivery
Purchase Bill	Quotation + Order + GRN
Sales Return / Purchase Return	Bill
Non-saleable Return (sales / purchase)	Bill

Each pulled line keeps its `source_*_detail_id`, so the child knows where it came from, and stock is never moved twice (a bill line from a delivery/GRN skips the stock movement).

The Quotation No. / Order No. / Challan (GRN) No. boxes sit in the master grid right after Customer / Vendor.

`pull()` also returns the source's master part (**MASTER** list: party sub-ledger, product company, agent, warehouse, area, route, cost center, unit, remarks, narration, rate type, currency / rate, account ledgers, party address ...), the document billing terms (`document_billing_terms` -> `header.billing_term_ids`), and the line terms (sales `line_terms`; purchase `document_line_billing_terms` -> `line.billing_term_ids + term_values`). `mergePulled()` writes these over the entry's values (only keys the form has); term id lists are merged.

5.2 Progress counters

`utils/progressCounters.js`:

- `moveSourceProgress(lines, spec, dir)` adds (`dir = +1` on posting) or subtracts (`-1` on cancel) the child quantity to the source line counter (`qty_ordered`, `qty_delivered`, `qty_received`, `qty_billed`, `qty_returned`, with `alt_` versions). It never goes below zero.
- `rollHeaderStatus` then sets the source header to none / partial / full (for example, an order becomes partially delivered).
- Pending lists show only what is still open (quantity minus counter).

6. Billing terms

6.1 Master (`billingTermRoutes`, `BillingTermManagement.jsx`)

A term has:

- Sign (add / deduct) and basis (value or quantity).
- Calculation: percentage, fixed rate, or a safe formula (`utils/formulaEvaluator.js` - no `eval`; variables `{basic_amount}`, `{quantity}`, `{rate}`, `{running_total}`, `{term:CODE}`).
- An optional cap and display order.
- A ledger and sub-ledgers: Billing, Return and Expiry Return ledger, each with its own default sub-ledger (`termSubLedgers`: expiry sub-ledger on non-saleable returns, return sub-ledger on sales / purchase returns).
- `tax_type`: none, vat or excise.
- Category, including `rounded_off`, which rounds the running total by `rounding_method` / `precision`; the term carries the difference.
- `entry_input_mode`: which of %, rate x qty, and amount the user may type.
- `manual_override`: whether the value may be changed in the entry.
- `show_in_term_summary`.
- `tds_applicable` (migration 151): on Purchase Additional the client's `tdsBase` takes only these terms' lines as the TDS base when any is present.

6.2 Evaluation

- `formulaEvaluator.evaluateAllTerms` evaluates the ordered term list for a line or a document.
- `utils/termInput.js` applies what the user typed:
 - `cleanInput`, `effectiveInput` and `applyInput`: a typed rate or amount replaces the computed value and the % is derived back for display.
 - `loadProductTermMap` gives per-product term values from the Product Master.
- Product-wise terms are stored in `document_line_billing_terms`, document terms in `document_billing_terms`.
- System Control `product_term_transactions` lists the documents whose grid shows product-wise terms. Product terms are still calculated for the others.
- The client mirrors this in `components/entry/lineCalc.js` (`termFromInput`, `productLineTerms`, `withTermValue`).

6.3 Editable gross / net

`components/entry/AmountCell.jsx`: typing a gross or net amount works out the rate, or the quantity when the product has the qty-from-amount flag for sales or purchase. `grossForNet` inverts the term chain by probing at K = 1,000,000.

7. Numbering, masters, currency

7.1 Document numbering (`utils/documentNumbering.js`, migration 139)

- A category is picked per document type, branch, user and fiscal year, within `valid_from` / `valid_to`.
- Modes:
 - `auto`: running number.
 - `manual_numeric` / `manual_alpha`: typed.
 - `auto_datewise` / `auto_monthwise`: the counter restarts per day or month (`period_key`), with a date part in Nepali or English format.
- Prefix, suffix, fill character, body length, `max_length`, `flexible_length`.
- The next number comes from the RPC `next_document_number_p`, which locks the category (advisory lock) so two users never get the same number.
- `previewDocumentNumber` shows the next number on the entry without using it.

7.2 Master codes (`utils/masterCodes.js`)

- Codes are generated automatically per master (e.g. `CUS`, `SUP`, `BTH` for ledgers by type, `LGR` for account groups), with a total-length limit.
- The Short Name stays editable.
- `utils/masterExtras.js` validates extra fields of Product Group, Product Company and Salesman/Agent.

7.3 Multi-currency (migration 141, `currencyRoutes.js`)

- The `currencies` table holds code, symbol and rate; NPR is created as the base currency on first read.
- Fourteen document tables carry `currency` and `exchange_rate`. The entry shows the Local Amount (amount x rate) in the term popups and totals.
- Current limit: GL posting uses the document amount as entered. It is not yet converted to local currency, so use NPR for posted documents until that conversion is added.

8. Settings that change behaviour

8.1 System Control (`systemControlRoutes`, `SystemControlSettings.jsx`)

One row per company in `system_control_settings`:

- Business Nature and feature flags (poultry, broiler, hatchery, construction, automobile), which show or hide menus (`menu.ts featureOn`).
- Default ledgers: sales, purchase, returns, VAT, GRN clearing, stock transfer/adjustment, non-saleable, rounding.
- Stock valuation method, negative stock control, batch/serial.
- Dual UOM mode, multiple warehouses, grid columns (free qty, batch, alt unit, mfg/exp).
- Bill-wise tracking default, credit control default.
- `approval_modules`, `product_term_transactions`.
- Computerized billing (IRD) and CBMS credentials.

8.2 Entry Field Control (`entryFieldControlRoutes`, `utils/entryFieldRules.js`)

Per document type (and optionally per user):

- Hide or show, compulsory, default value, and locked fields.
- Which source documents may be pulled.

`checkCompulsoryFields` and `lockProtectedFields` enforce these on the server.

8.3 Security and data access

- **Security Rights Group** (`securityGroupRoutes`): `permissions[module][view|create|edit|delete|print]` and `permissions.approvals[type]`.
- **Data Access** (`utils/dataAccess.js`): per user or group, the ledgers, sub-ledgers, products, companies, groups, categories and areas a user may see. `guard` filters every JSON response and refuses saves that use hidden ids. Postings are never filtered. Company admins are not restricted.
- **Super Admin view only** (`middleware/auth.js`): a super admin token with a tenant (`switch-tenant`) is `readOnly`; `readOnlyBlocked` refuses every non-GET call except `READ_ONLY_ALLOW` (auth, admin, company create, previews, pending pull, UDF lookup, print log). `routes/adminRoutes.js` (`/admin/overview`, `/admin/tenants/:id`), `POST /auth/exit-tenant`, `pages/AdminPanel.jsx`; `PrivateRoute` sends a super admin without a company to `/admin`.
- **Firm import** (`utils/firmImport.js`, `routes/firmImportRoutes.js`, migration 155 `firm_import_log`): sources = `getUserTenants` of the user; masters in dependency order matched by code, every `*_id` / `*_ids` UUID translated through one source-id -> target-id map (unknown -> null, same-list references filled after), opening columns zeroed unless asked; transactions (with lines and document / line billing terms) inserted as drafts per document all-or-nothing.

9. Reports

- **Financial** (`financialReportRoutes` -> `financialEngine`): Trial Balance, P&L, Balance Sheet, notes, ratios, cash flow and funds flow, all from one engine.
- **Ledger / Party / Day Book / Cash-Bank**: `ledgerReportRoutes`, `partySummaryRoutes`, `controlReports`.
- **Stock**: `stockReportRoutes` -> `stockEngine`, `stockReport`, `stockInOut`, `stockHealth`, `analytics` (FSN, ABC, XYZ, forecasts).
- **VAT / IRD**: `vatReportRoutes` -> `vatLedger`; `ird.js` for the materialized register and CBMS sync.
- **Dimensions / budgets**: `dimensionReports`, `budgetReports`.
- **Sales / purchase analysis**: `tradeAnalysis`, `tradeLines`, `salesman`, `agentTargets`.
- **Daily Register** (`utils/dailyRegister.js`, `GET /api/daily-register`, `DailyRegister.jsx`): built on `dayBook` rows; A-J totals, cash check, fiscal-year-to-date sales / purchase and `financialEngine.profitAndLoss`. With a user / agent filter the ledger-based figures are null.
- **Consignment costing** (`utils/consignmentCost.js`): `allocateAdditional(raw, addl)` spreads each Purchase Additional line to bill lines - product-wise target wholly to its line, else by basis (value net of VAT / qty / equal, last line takes rounding); non-costing lines (TDS, basis none) and claimable VAT go to `non_additional`. Purchase Additional links only to a Ref. Bill (`source_bill_id` required on post).
- **Purchase Additional customs** (migrations 152-153, `purchase_additional_customs`): VAT-report only, never posted. `normCustoms` sums item-wise rows (`detail_mode`, `item_details` jsonb); `vatReportRoutes.loadCustomTaxDocs` adds each row as an import bill (`is_import`, `import_taxable`, item lines). Customs Offices master: `customsOfficeRoutes` + `utils/customsOffices.ensureDefaults` (Nepal's offices, once per company). Bill type of a line: `billTypeOf`.
- **Advanced VAT reports** (`utils/vatAnnex.js`: `annex13`, `monthlySalesPurchase`, `partySummary`; routes `/vat-reports/annex13|monthly-sales-purchase|party-summary`; client `components/vat/VatAdvancedReports.jsx`): all from `loadTaxDocs`, include flags per kind (`kinds(q)`), nature split by product `item_type`, balances from `financialEngine.ledgerBalances`. `loadTaxDocs` rows now carry `party_code`, `party_sub_ledger_id`, `product_company_id`, `currency`, `is_export`.
- **Auto TDS** (migration 154: `billing_terms.tds_percent`, lines `tds_percent` / `tds_base`, header `auto_tds`): `PurchaseAdditionalExpense.jsx` `lineTds` / `buildTds` / `withTds` rebuild one TDS line per (party, rate) in an effect on every relevant change.
- **JV view** (`components/entry/PostingView.jsx`, `GET /api/document-posting/:id` in `entryHelperRoutes`): every ledger batch of a document id; the "JV" button of `DocActions` (all transaction lists) and PDC.
- **Report views & table filters** (`hooks/useExcelTableFilters.tsx`, `components/ReportViews.jsx`): leaf header cells read as a grid (`colSpan` / `rowSpan`); state per table = filters, sort, hidden columns; `captureTables` / `restoreTables` (pending until a table with the same header signature appears). `ReportViews` stores `{tabs, fields, tables, modes, grids}` under report key `page:<path>` in `saved_report_views`, sets fields with the native value setter and presses the page's Show / Run button.
- **Report grid on every report** (R21): `hooks/useSmartTables.jsx` (called by `Layout`) finds the same tables as the filter (`reportTables`, 2+ data rows, not in a pop-up, not on an entry screen with `.ent-fillbar`, opt out `data-no-smart`), inserts a `.sg-mount` before the table (inside its wrapper when the wrapper sits in a grid / flex row) and portals `SmartTable` into it. `extractTable(table)` reads captions (grouped headers "A > B", caption-only header rows ignored), data rows (numbers via `toNum`, "Cr" = minus; identifier columns - PAN, bill no, code - stay text), heading / bold tree rows -> level columns `__sec1..n` (level from `padding-left`, named from "Customer > Product Group > Product"), opening / total rows -> `pinnedTop` / `pinnedBottom`, sub-totals between rows dropped; `simple` decides Grid vs Report first. Cells keep their text (`c0__t`) and actions (`c0__a`): a grid click clicks the same link / button / row in the hidden report. Mode per report in `localStorage` `sg_mode:<path>:<hash>`; print CSS shows the report (`[data-sg-hidden]`).

- **ReportGrid** (`components/ReportGrid.jsx`): multi-level sort, groups with per-level order (`groupSort`), pivot areas (`groupByKeys = Rows, pivotCols, pivotValues, filterFields, showGrouped`) -> `grid/PivotView.jsx` via `gridAnalysis.buildPivot` (tree x column tuples x measures, accumulators merged so Average / Min / Max totals stay right), `grid/FieldSelector.jsx` (areas, drag payloads `col:<key> / area:<area>:<key>`), `grid/GridChartPanel.jsx` + `charts/DataChart.tsx` (multi-series SVG, `buildChartData`: split, Top N + Other), data bars, added columns (`calcColumns`), footers (`aggregate / fmtAgg`), toolbar pop-ups portalled to body (`Pop`). Layout state in `localStorage <storageKey>_state`; the full state (with filters) through `grid/gridRegistry.js` for Views (`config_json.grids, config_json.modes`). Pure logic tested in scratch `r41`.
- **Universal register**: `registerRoutes` with `documentCatalog.js`, the list of every document type and its table.

10. Industry modules

Shown only when System Control turns them on.

- **Poultry** (`poultryRoutes`, `utils/poultry.js`, `utils/hatchery.js`): sheds, batches, daily log (feed, medicine, mortality), lifting (sale), hatchery (egg set, candling, hatch). Consumption and lifting reuse stock adjustment and sales postings, so stock and GL follow the rules above.
- **Construction** (`constructionRoutes`, `utils/construction.js`): sites/contracts, running bills, sub-contracts, material issue and wages per site. Site profit = billed - material - wages - sub-contract.
- **Automobile** (`automobileRoutes`, `utils/automobile.js`): enquiries, vehicle stock/PDI/delivery, job cards, parts issue, outside work and service reminders.

11. Adding a new transaction module - checklist

1. Write a migration: header + details tables with `tenant_id`, `status`, `doc_no`, `currency`, `exchange_rate`, snapshot columns, and `source_*_detail_id` on the lines. Run `audit_attach_all()`.
2. Write the route: list/get/create/update, `PUT /:id/status` with guarded post and reverse, `DELETE` for drafts only. Use `resolveDocumentNumber`, `checkCompulsoryFields`, `splitByAccount`, `postBatch/reverseBatch`, and stock movements with base-unit conversion.
3. Add the API to `approval.js` `API_TYPES` and to `DocActions` `API_TYPE`, and a label to `approvalDocs.js`.
4. Register the document in `documentCatalog.js` so the Universal Register sees it.
5. Build the page with `EntryFillBar`, the line grid, term popups, `CurrencyField` and `DocActions`; add a route in `App.js` and a menu entry in `menu.ts`.
6. Add a test in `r5/`, document the posting rule in section 3.4, and add the menu to `USER_MANUAL.md`.

12. Running and testing locally

- Apply migrations in order: `psql ... -f database/NNN_*.sql` (each file can be re-run).
- Server: `cd server && npm install && node server.js` (port 5000).
- Client: `cd client && npm install && npm start` (port 3000).
- Server tests: `node r5/<name>_test.js` (each prints PASS/FAIL).
- Client checks: `npx eslint src, npx tsc --noEmit -p ., CI= npx react-scripts build`.
- Rebuild the PDFs: `python3 docs/tools/build_pdfs.py` (needs reportlab).
- In-system help: edit `docs/tools/help_fields.py` (caption -> what it does), `help_pages_entry.py` / `help_pages_other.py` (per screen: purpose, steps, effect), then run `python3 docs/tools/build_help.py`. It writes:
 - `client/public/help/helpCatalog.json`, loaded on demand by `components/help` (? Help panel of every screen, caption tooltips through `useFieldTips` in `Layout`);
 - one PDF per menu screen in `client/public/help/menus/` - the field list of a screen is read from its page source (`erp-label` captions);
 - the manuals (complete, per Business Nature, IRD architecture / user manual, developer guide, audit) in `client/public/help/`, with copies in `docs/pdf/`;
 - `client/public/help/index.json` for the Help Center (`/help-center`, `pages/HelpCenter.jsx`). The developer guide, the audit and the menu PDFs are shown to super admin only.